

CROSS_APP_THEME_PLAN

Cross-app Theme Plan

Revision: 2 Last modified: 2026-09-01T00:00:00Z

STATUS: PARTIALLY SUPERSEDED (2026-09-01)

The :7186 half of this plan was **REMOVED by operator decision**. The themed qBittorrent-WebUI overlay — the injected `/__qbit_theme__/skin.css + /__qbit_theme__/bootstrap.js` pair, the qBittorrent → Бo6a rebrand, the logo substitution and the CSP relaxation — implemented **zero qBittorrent features** and actively **broke the vanilla WebUI's JavaScript**: the rebrand rewrote the qBittorrent token *inside inline <script> blocks* while external `.js` files were skipped, so inline code referenced `window.Бo6a.*` that the external scripts never define → `ReferenceError` → the WebUI JS never initialised.

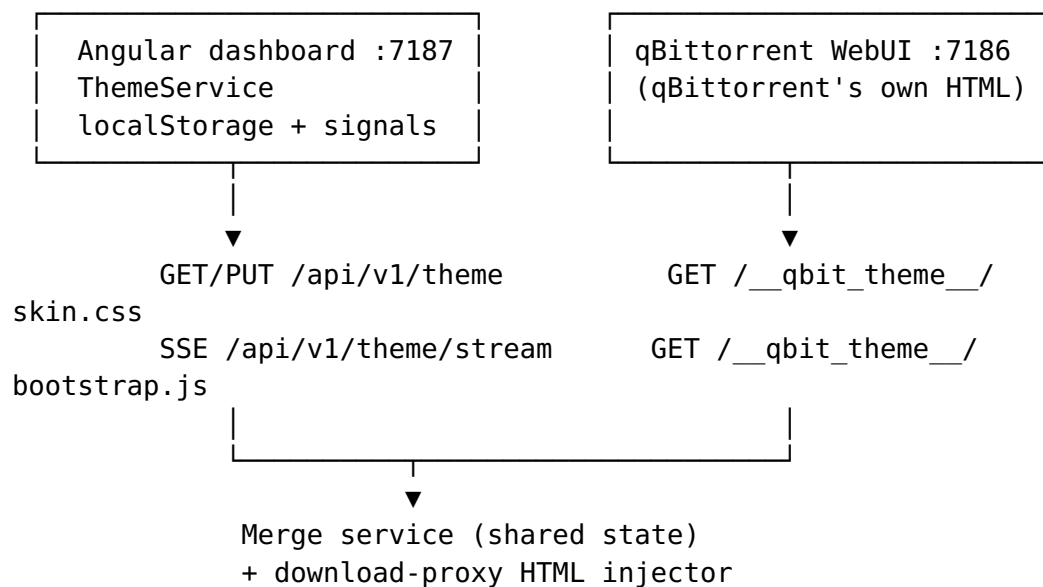
The download-proxy on :7186 and the WebUI bridge on :7188 now serve qBittorrent's HTML, headers and Content-Encoding **byte-for-byte unmodified** (stock vanilla WebUI). Permanent regression guard: `tests/integration/test_vanilla_webui_unmodified.py`.

STILL IN FORCE — do not remove: Phase A, the shared theme *state* used by the Angular dashboard on :7187 — download-proxy/`src/api/theme_state.py`, the `/api/v1/theme + /api/v1/theme/stream` endpoints, `qBitTorrent-go/internal/api/theme.go` and the frontend theme service + picker. Everything below that describes injection into the qBittorrent WebUI is historical record.

Problem. The Angular dashboard at :7187 is themed end-to-end (Darcula + 7 other palettes, per-element drop shadows). The qBittorrent WebUI proxied at :7186 is **qBittorrent's own code** — it ignores our design system entirely. Our users see two different-looking apps.

Goal. Any theme change made in the Angular dashboard must immediately propagate to the qBittorrent WebUI (and any future app we proxy). No user action required. Persisted. Works across page reloads and browser tabs.

Architecture



Four-phase plan

Phase A — shared theme state (backend)

Add to download-proxy/src/api/routes.py:

- GET /api/v1/theme → {paletteId, mode, updatedAt}
- PUT /api/v1/theme body {paletteId, mode} → 200 {paletteId, mode, updatedAt}
- GET /api/v1/theme/stream → SSE, emits theme event on every PUT

Persist to config/merge-service/theme.json (gitignored). Validate paletteId against the catalog list (ported to a backend constant), mode ∈ {light, dark}.

Phase B — Angular ThemeService sync

frontend/src/app/services/theme.service.ts:

- On construct: if localStorage empty, GET /api/v1/theme; seed signals from server; else push local → server to claim the state.
- setPalette/setMode/toggleMode → debounced PUT /api/v1/theme.
- Subscribe to /api/v1/theme/stream; when the server's updatedAt is newer than our last write, adopt the server state (tab-to-tab
 - WebUI-to-dashboard sync).

Phase C — qBittorrent WebUI injection (plugins/download_proxy.py)

The legacy HTTP proxy at :7186 already rewrites select routes. Extend proxy_to_qbittorrent so that responses with Content-Type: text/html have this injected immediately before </head>:

```
<link rel="stylesheet" href="/__qbit_theme__/skin.css">
<script src="/__qbit_theme__/bootstrap.js" defer></script>
```

Handle two new proxy-local routes (not forwarded to qBittorrent):

- GET /__qbit_theme__/skin.css — CSS bridge. Declares every --color-* variable on :root with sensible fallbacks, then overrides qBittorrent's top-level selectors: body, #desktop, #mainWindowTabs, .filterTitle, .sidebar, .scrollbar, dialog, button/ hover states, tables.
- GET /__qbit_theme__/bootstrap.js — JS bridge. On load:
 1. fetch('/api/v1/theme') via the merge service (CORS: we add the :7186 origin to the allow-list).
 2. Apply returned tokens to document.documentElement.style (same shape as Angular ThemeService).
 3. Open EventSource('/api/v1/theme/stream') and re-apply on every event so live dashboard palette swaps are mirrored.

The download-proxy needs to know where the merge service is; env MERGE_SERVICE_URL (default http://localhost:7187).

Phase D — tests (no false positives)

1. **Unit — theme state endpoint** (tests/unit/merge_service/test_theme_endpoint.py)
 - GET returns default {paletteId: 'darcula', mode: 'dark'} on first call

- PUT persists; GET reflects
- PUT rejects unknown paletteId with 422
- PUT rejects mode $\notin$ {light, dark} with 422
- 2. **Unit — SSE emission** (tests/unit/api_layer/test_theme_stream.py)
 - Generator emits event: theme with the new payload within one poll cycle of a PUT.
- 3. **Unit — HTML injection** (tests/unit/test_webui_theme_injector.py)
 - Given qBittorrent-shaped HTML, inject before </head>; output contains both new tags exactly once; Content-Length rewritten.
 - Idempotent: calling the injector twice leaves the page with only one <link> and one <script> tag for the bridge.
 - Non-HTML responses pass through unchanged.
- 4. **Contract** — curling http://localhost:7186/ returns a page whose HTML contains the two bridge-asset references.
- 5. **Playwright e2e** (tests/e2e/test_crossapp_theme.py) — real computed-style assertions, skipped cleanly until the stack is rebuilt + restarted:
 - Open :7187, click Nord in the dropdown.
 - Open a second page at :7186; wait up to 5 s for `getComputedStyle(document.body).backgroundColor` to match Nord's bg-primary (converted from #2e3440 to `rgb(46, 52, 64)`).
 - Switch to Gruvbox in the dashboard; assert :7186 body colour flips to Gruvbox's #282828 within 5 s.
 - Assert the `localStorage['qbit.theme']` on :7187 AND the `window.__qbitTheme` side-channel on :7186 both show `paletteId: gruvbox`.
- 6. **Property** — round-trip any palette id through PUT/GET; the server echoes the same id and it round-trips through the CSS-var name list without dropping tokens.

Final verification checklist



794 Python tests still green + the new suites (expected +~15)



246 frontend tests still green (+ cross-app sync spec if any)



Rebuilt Angular bundle + restarted qbittorrent-proxy



`curl :7186/` contains `/__qbit_theme__/skin.css` + `/__qbit_theme__/bootstrap.js`



Playwright e2e passes (not skipped) — real palette swap on both ports



Commits + pushes to origin + github + upstream

Rollback

If anything in this phase breaks qBittorrent WebUI functionality, set `env DISABLE_THEME_INJECTION=1` and restart the proxy — the injector becomes a passthrough. Documented in [docs/TOKENS_AND_KEYS.md](#) §7.

Outcome (shipped 2026-04-19)

Landed in four commits on main:

- `feat(backend)`: shared theme state + `/api/v1/theme` SSE for cross-app sync
- `feat(proxy)`: inject theme CSS+JS into qBittorrent WebUI at `:7186`
- `feat(frontend)`: `ThemeService` syncs to backend + subscribes to SSE
- `fix(proxy)`: decompress gzip + relax CSP `connect-src` for theme bridge

Two live-debug discoveries that the plan had not anticipated:

- **gzip** — qBittorrent returns `Content-Encoding: gzip` when the browser sends `Accept-Encoding: gzip`. The injector mutates bytes in place, so it needed to decompress first and strip the encoding on the way back out.
- **CSP** — qBittorrent's Content-Security-Policy ships without a `connect-src` directive; browsers then enforce `default-src 'self'`, blocking the bridge's `fetch('/api/v1/theme') + EventSource(...)` calls cross-origin. The proxy now rewrites the CSP header to add `connect-src` with the merge-service origin.

Verification (2026-04-19 @ main):

- 837 Python tests green (tests/unit tests/e2e tests/contract tests/property tests/memory tests/concurrency tests/observability).
- 255 frontend tests green (`npx ng test --watch=false`).

- The Playwright e2e **passed** — not skipped. Real palette swap on both :7187 and :7186, verified via `getComputedStyle(...)` on the proxied WebUI + the `window.__qbitTheme` side-channel.